

1교시: 1주차 복습 및 Docker 학습 목표

수업 목표

- Week 1에서 만든 로컬 실행 evidence를 Docker 학습의 출발점으로 연결한다.
- Docker가 해결하려는 문제를 "명령어 도구"가 아니라 실행 환경 표준화 문제로 설명한다.
- 오늘 남길 Docker readiness evidence의 기준을 이해한다.

선행 지식

이미 알고 있어야 할 것	오늘 다시 연결할 것
local folder, start command, port, browser/curl 확인	Docker image, container, port binding의 필요성
README run step	Docker run/build/cleanup section
log/status evidence	docker logs, docker ps, HTTP response
blocker/RCA note	Docker 설치/실행 실패 기록

50분 흐름

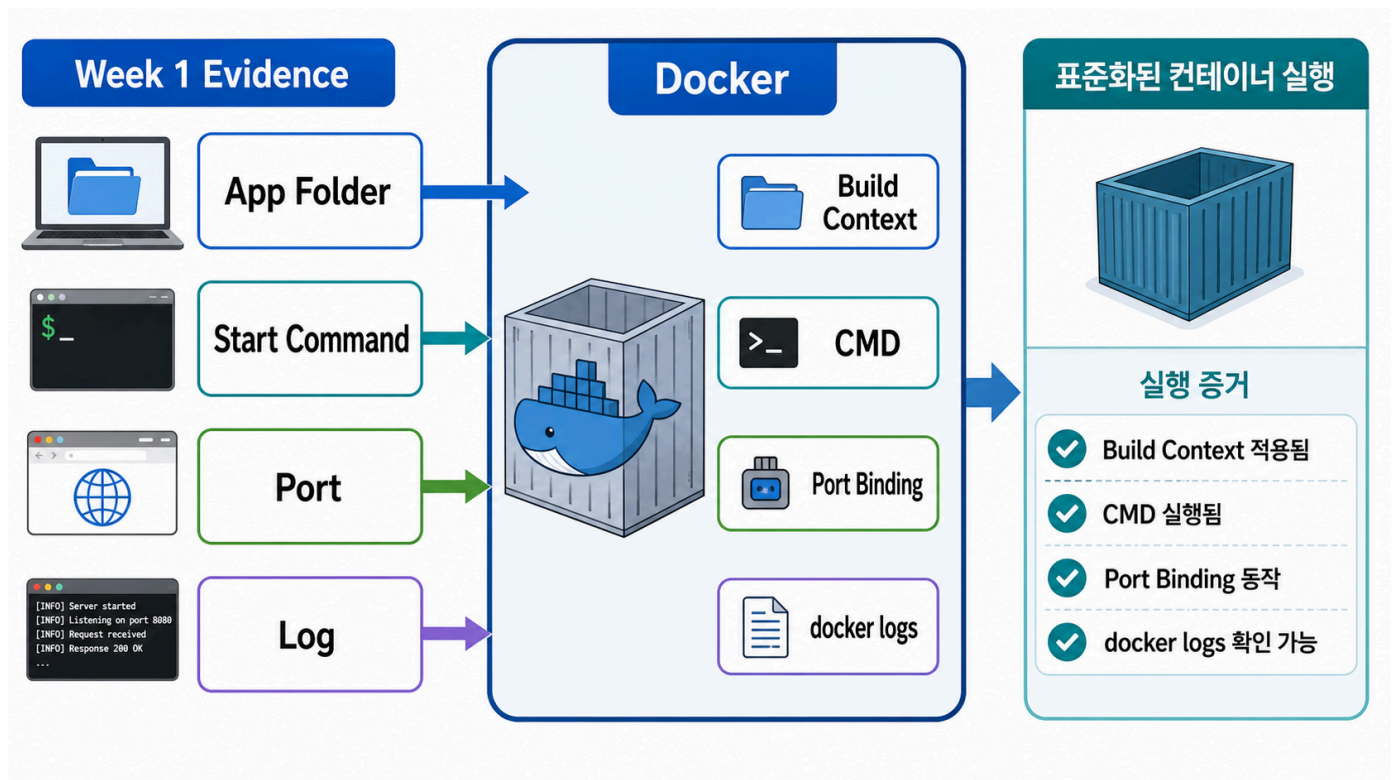
시간	활동	학습 초점	학생 산출
0-5분	Week 1 산출물 빠른 확인	로컬 실행 조건을 기억이 아니라 evidence로 본다.	Week 1 evidence 위치
5-15분	"내 컴퓨터에서는 되는데" 문제 분석	환경 차이, 의존성, port, path 문제를 분류한다.	문제 분류 표
15-25분	Docker가 표준화하는 것과 표준화하지 않는 것	image/container가 해결하는 범위와 한계를 구분한다.	Docker 학습 목표 문장
25-35분	Docker lifecycle preview	run, check, logs, stop, rm의 순서를 이해한다.	lifecycle note
35-45분	Day 1 evidence 양식 작성	설치 성공/실패 모두 기록 가능한 상태로 만든다.	docker-evidence.md 초안
45-50분	다음 교시 설치 준비	OS, 권한, 공식 문서 링크를 확인한다.	설치 준비 checklist

0-5분 Week 1 산출물 빠른 확인

Week 1에서 만든 미니 앱은 Docker 실습의 배경 자료다. 오늘 당장 개인 앱을 모두 container로 바꾸지는 않는다. 먼저 그 앱이 실행되기 위해 어떤 조건이 필요했는지 확인한다.

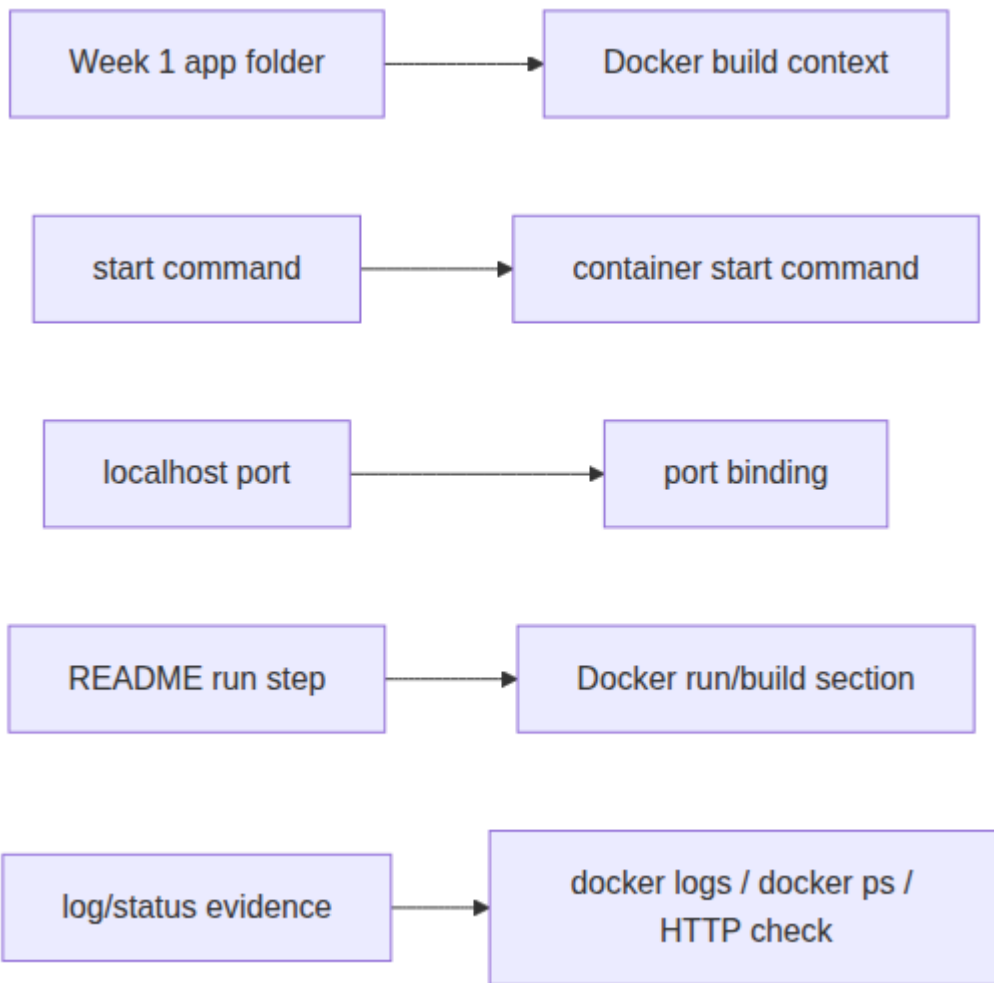
현업에서 "실행됩니다"라는 말은 충분하지 않다. 어느 directory에서 어떤 command를 실행했고, 어떤 port로 접근했고, 어떤 상태값이나 로그로 정상임을 확인했는지 같이 남아야 한다. Docker는 이 조건을 image와 container로 포장하지만, 원래 조건이 불명확하면 Dockerfile도 불명확해진다.

Visual 1: Week 1 evidence에서 Docker 실행 조건으로



이 이미지는 Week 1의 app folder, start command, port, log가 Docker의 build context, CMD, port binding, docker logs 로 옮겨지는 관계를 보여준다. 왼쪽에서 오른쪽으로 읽으며 Docker가 기존 실행 조건을 새로 만드는 것이 아니라, 이미 확인한 실행 조건을 표준 실행 단위로 포장한다는 점을 확인한다.

Visual 2: Docker 실행 조건 Mermaid map



읽는 순서: 왼쪽의 Week 1 evidence를 오른쪽 Docker 표현으로 연결한다. 이 visual은 Docker가 새로운 마법을 추가하는 것이 아니라, 기존 실행 조건을 더 명시적인 실행 단위로 옮긴다는 점을 보여준다.

5-15분 "내 컴퓨터에서는 되는데" 문제 분석

Docker를 배우는 첫 이유는 개발자가 만든 코드를 운영 환경에서 재현 가능하게 실행하기 위해서다. 같은 소스코드라도 runtime version, package, path, port, environment variable, permission이 달라지면 결과가 달라진다.

Docker image는 실행에 필요한 파일, binary, library, configuration default를 하나의 표준 패키지로 만든다. container는 그 image에서 시작된 실행 중인 process다. 따라서 Week 1의 process, filesystem, network 개념은 사라지지 않는다. Docker 안에서 이름과 경계가 달라질 뿐이다.

Visual 3: 환경 차이 문제 분류

문제 증상	Week 1 표현	Docker에서 다른 표현	확인 evidence
내 PC에서는 되지만 다른 PC에서는 실행 실패	runtime/package 차이	base image, image layer	Dockerfile, build log
browser 접속이 안 됨	localhost, port, HTTP status	host port, container port	docker ps, curl
데이터가 사라짐	file path, persistence	bind mount, named volume	volume list, app log
설정이 바뀌지 않음	config file, env var	-e, .env, Compose environment	container env, README

원인을 모르겠음	log/status 부족	docker logs, status, exit code	RCA note
----------	---------------	--------------------------------	----------

이 표를 볼 때 "Docker가 있으면 문제가 없어지는가"가 아니라 "어떤 종류의 문제를 더 관찰 가능하게 만들 수 있는가"를 질문한다.

15-25분 Docker가 표준화하는 것과 표준화하지 않는 것

Docker는 실행 환경 표준화에 강하지만, 모든 운영 책임을 자동으로 해결하지 않는다. image에 secret을 넣으면 secret은 더 넓게 복제될 수 있고, container를 많이 실행하면 host disk와 memory를 사용한다. latest tag만 사용하면 시간이 지난 뒤 같은 명령이 다른 image를 가져올 수 있다.

따라서 Docker 학습은 비용 절감, 개발/배포 효율, 관리 효율을 동시에 다룬다. 비용 절감은 로컬에서 빠르게 재현하고 불필요한 cloud resource 생성을 미루는 데서 시작한다. 개발/배포 효율은 실행 조건을 image로 고정해 handoff 시간을 줄이는 데서 나온다. 관리 효율은 run/check/stop/cleanup 흐름을 README에 남길 때 생긴다.

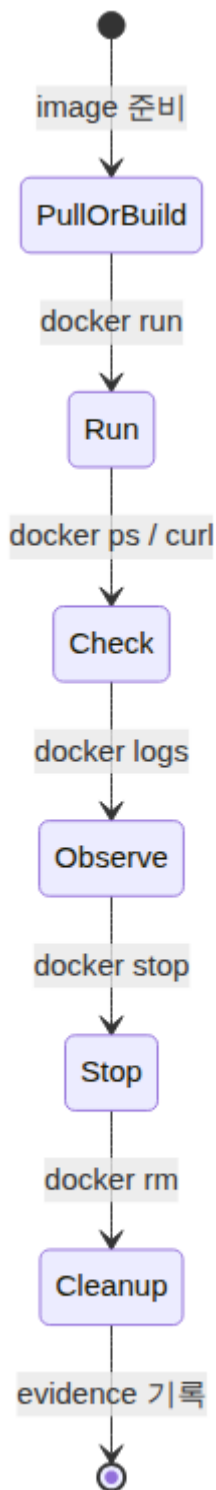
판단 표: Docker를 쓰기 전 확인할 질문

질문	Docker가 도움이 되는 경우	주의해야 할 경우
실행 조건이 반복되는가?	같은 앱을 여러 장비에서 실행해야 함	일회성 script라 포장 비용이 더 큼
의존성이 복잡한가?	runtime/package 차이가 자주 발생	image가 커져 build/pull 시간이 늘어남
외부 설정이 필요한가?	env var로 설정을 주입할 수 있음	secret을 image에 넣는 실수를 할 수 있음
데이터가 남아야 하는가?	volume으로 lifecycle을 분리할 수 있음	volume 초기화/삭제 실수로 데이터 손실 가능
누가 이어받는가?	README와 Dockerfile로 handoff 가능	Dockerfile이 불명확하면 문제도 포장됨

25-35분 Docker lifecycle preview

오늘 반복할 기본 사이클은 실행, 확인, 관찰, 중지, 정리다. 컨테이너를 실행만 하고 정리하지 않으면 port가 계속 점유되거나 disk가 불필요하게 쌓인다. 운영에서는 "시작했다"만큼 "어떻게 멈추고 원상복구하는가"도 중요하다.

Visual 4: 기본 lifecycle



읽는 순서: 컨테이너 실습은 `run` 에서 끝나지 않는다. 정상 확인과 로그 확인, 중지, 삭제, 기록까지 끝나야 하나의 운영 사이클이 닫힌다.

35-45분 Day 1 evidence 양식 작성

아래 양식을 `docker-evidence.md` 또는 README의 Docker 섹션에 작성한다. 설치가 실패해도 빈칸으로 두지 않는다. 실패한 단계, error message, 시도한 공식 문서 링크가 있으면 다음 도움 요청이 빨라진다.

```
# Docker Day 1 Evidence
```

Environment

- OS:
- Docker Desktop status:
- Docker version command:
- Result summary:

First Run

- Command:
- Image:
- Expected result:
- Actual result:
- Browser/curl evidence:

Cleanup

- Stop command:
- Remove command:
- Remaining blocker:

Safety

- Secrets exposed? no / needs review
- Screenshot filenames:

흔한 오해

오해	바로잡기
Docker는 VM과 같다.	container는 보통 host kernel 위에서 격리된 process로 실행된다. VM과 같은 방식으로 OS 전체를 매번 부팅하는 모델이 아니다.
image와 container는 같은 말이다.	image는 실행 패키지이고 container는 image에서 시작된 실행 상태다.
설치가 안 되면 수업을 따라갈 수 없다.	설치 실패도 OS, 권한, error evidence가 있으면 해결 가능한 blocker가 된다.
Docker를 쓰면 README가 덜 중요하다.	Docker 명령과 cleanup 절차를 README에 남겨야 다른 사람이 같은 결과를 만든다.

45-50분 다음 교시 설치 준비

다음 교시는 Docker Desktop 설치와 계정 상태 확인이다. 기본 실습은 macOS 기준으로 진행하므로 Mac 설치 문서에서 Apple silicon/Intel 구분, system requirement, 권한 조건을 확인한다. Windows 장비를 쓰는 학생은 별도 예외 경로로 Windows 설치 문서의 WSL 2와 virtualization 조건을 확인한다.

평가 기준

기준	2점 evidence
Week 1 연결	app folder, command, port, log 중 최소 2개를 Docker 학습 목표와 연결했다.
문제 분류	환경 차이, port, 설정, log 중 하나 이상을 구체 증상으로 분류했다.
lifecycle 이해	run/check/logs/stop/rm 순서를 설명했다.
보안 책임	Docker Hub credential, token, MFA code를 기록하지 않는다고 명시했다.
다음 준비	OS별 설치 공식 문서와 blocker 기록 양식을 준비했다.

공식/학술 근거 링크

- Docker Docs: Docker overview, <https://docs.docker.com/engine/docker-overview/> - image, container, registry, Docker Engine의 공식 개념 기준이다.
- Docker Docs: What is an image?, <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/> - image가 실행 파일, library, configuration을 포함하는 표준 패키지라는 기준이다.
- Google SRE Book: Postmortem Culture, <https://sre.google/sre-book/postmortem-culture/> - 실패를 숨기지 않고 evidence와 follow-up으로 바꾸는 운영 문화의 근거다.

다음 연결

다음 교시에는 Docker Desktop 설치 상태를 확인한다. 성공한 학생은 `docker version` 과 `hello-world` 준비로 넘어가고, 막힌 학생은 macOS 권한/실행 상태 또는 Windows WSL 2/가상화 조건, error message를 blocker evidence로 정리한다.